

Indian Institute of Information Technology Bangalore

Department of Computer Science & Engineering

Cloud-Native Financial System: Microservices with GitOps-Driven DevOps Pipeline

Major Project Report

Software Production Engineering (SPE)

Submitted by:

G Sunil MT2025049

Mahendranath Gulla MT2025712

May 2026

1. Introduction

Modern financial applications demand high availability, independent scalability, and zero-downtime deployments. Monolithic architectures struggle to meet these requirements as codebases grow. This project addresses those challenges by designing and deploying a **cloud-native financial system** built entirely on microservices, with every aspect of its infrastructure managed through code and automated pipelines.

The system implements five core backend microservices — **User Service**, **Account Service**, **Transaction Service**, **Budget Service**, and **Notification Service** — each independently developed, tested, containerized, and deployed. Services communicate synchronously via OpenFeign HTTP clients and asynchronously through Spring events, following the **Saga Orchestration** pattern for distributed transactions.

On the infrastructure side, the project implements a complete **GitOps pipeline** using Jenkins for CI, ArgoCD for CD, Helm Charts for Kubernetes manifests, and HashiCorp Vault for secrets management. Observability is handled by the ELK stack, with structured logs shipped from every service into a centralized Kibana dashboard.

The scope of this report covers the backend microservices and the full DevOps infrastructure. The frontend web dashboard is outside the scope of this submission.

2. System Architecture

2.1. Architectural Overview

The system is organized into three distinct layers, each with a clearly defined responsibility:

- **Infrastructure Layer** — Manages the GitOps pipeline (GitHub, Jenkins, ArgoCD), the Kubernetes cluster provisioned via Helm Charts, and HashiCorp Vault for secret injection.
- **Microservices Layer** — Five Spring Boot services, each with its own PostgreSQL database, following the database-per-service pattern for true isolation.
- **Observability Layer** — ELK Stack collects structured JSON logs from all services and exposes them through Kibana.

Figure 1 shows the complete system architecture.

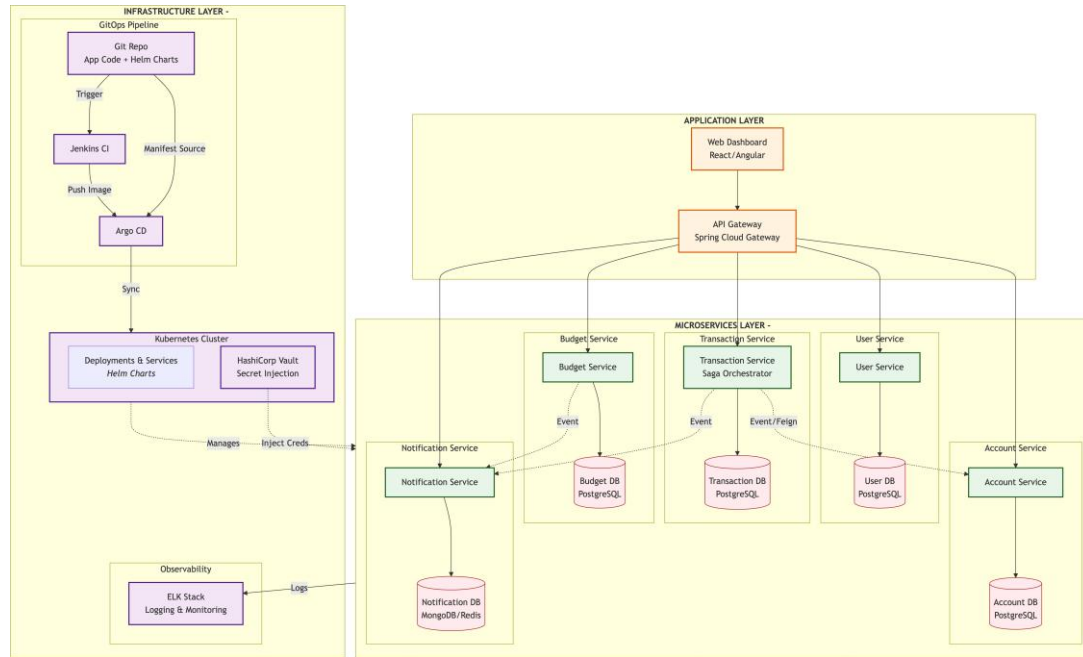


Figure 1: Complete System Architecture — Infrastructure, Microservices, and Observability Layers

2.2. End-to-End Pipeline Flow

The GitOps pipeline is the backbone of the deployment strategy. Figure 2 shows the sequence of events from a developer pushing code to the application running inside the Kubernetes cluster.

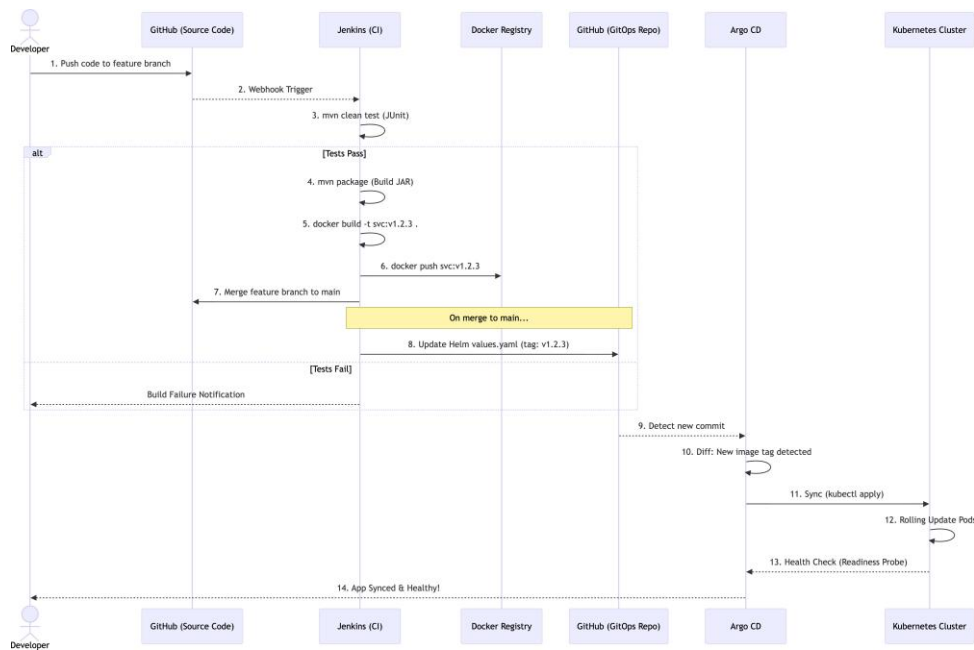


Figure 2: End-to-End GitOps Pipeline — Sequence Diagram

The flow can be summarized in the following steps:

1. A developer pushes code to a service repository on GitHub.
2. A webhook triggers Jenkins, which clones the repository and runs the CI pipeline.
3. Jenkins runs Maven unit tests (JUnit). If tests fail, the build is aborted and an email notification is sent to the team.
4. On test success, Jenkins packages the application into a JAR, builds a Docker image tagged with the Jenkins build number, and pushes it to DockerHub.
5. Jenkins then clones the GitOps repository and updates the `image.tag` value in the relevant Helm chart's `values.yaml` file, committing the change back to GitHub.
6. Ansible, triggered from the final Jenkins stage, logs into ArgoCD and syncs the application.
7. ArgoCD detects the new commit, diffs the desired state (Helm chart) against the live cluster state, and performs a rolling update of the affected pods.
8. Kubernetes performs health checks using readiness and liveness probes before routing traffic to the new pods.
9. At every stage, structured logs from all services flow into the ELK stack for monitoring.

2.3. Microservices and Their Responsibilities

Table 1: Microservices Summary

Service	Responsibility	Database
User Service	User registration, login, and profile management	PostgreSQL
Account Service	Bank account creation and balance management	PostgreSQL
Transaction Service	Fund transfers via Saga Orchestration, inter-service coordination	PostgreSQL
Budget Service	Budget creation, spend tracking, threshold alerts	PostgreSQL
Notification Service	Receives events from other services and dispatches notifications	MongoDB/Redis

2.4. Inter-Service Communication

Services communicate in two ways:

- **Synchronous (OpenFeign):** Transaction Service calls Account Service to validate and debit/credit balances. Account Service calls User Service to verify user existence.
- **Asynchronous (Spring Events):** Budget Service and Transaction Service publish events that Notification Service consumes to send alerts, decoupling the notification concern from business logic.

3. Source Code Management — GitHub

3.1. Repository Structure

All source code is hosted on GitHub under the financial-system-project organization. The organization contains six repositories: one per microservice and a dedicated GitOps repository that holds all Helm charts and ArgoCD application manifests. This clean separation ensures that application developers never need to touch infrastructure files directly — only the automated Jenkins pipeline writes to the GitOps repo.

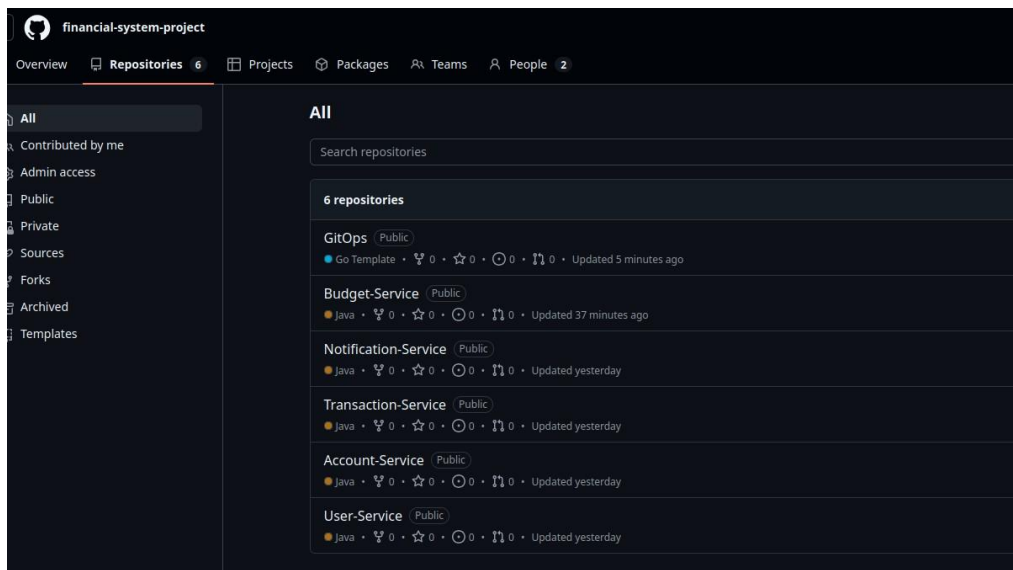


Figure 3: GitHub Organization showing all six repositories — five microservices and the GitOps repo

Each microservice repository follows a consistent structure: Java source code under `src/`, a `Dockerfile` for containerization, a `Jenkinsfile` defining the CI pipeline, and a `pom.xml` for Maven configuration.

3.2. Jenkinsfile — Declarative Pipeline Definition

Rather than configuring Jenkins pipelines through the UI, each service repository contains a `Jenkinsfile` that codifies the entire CI pipeline. This means pipeline changes go through the same code review process as application changes, and every build is perfectly reproducible. Figure 4 shows the `Jenkinsfile` from the User Service, which defines the eight-stage pipeline.

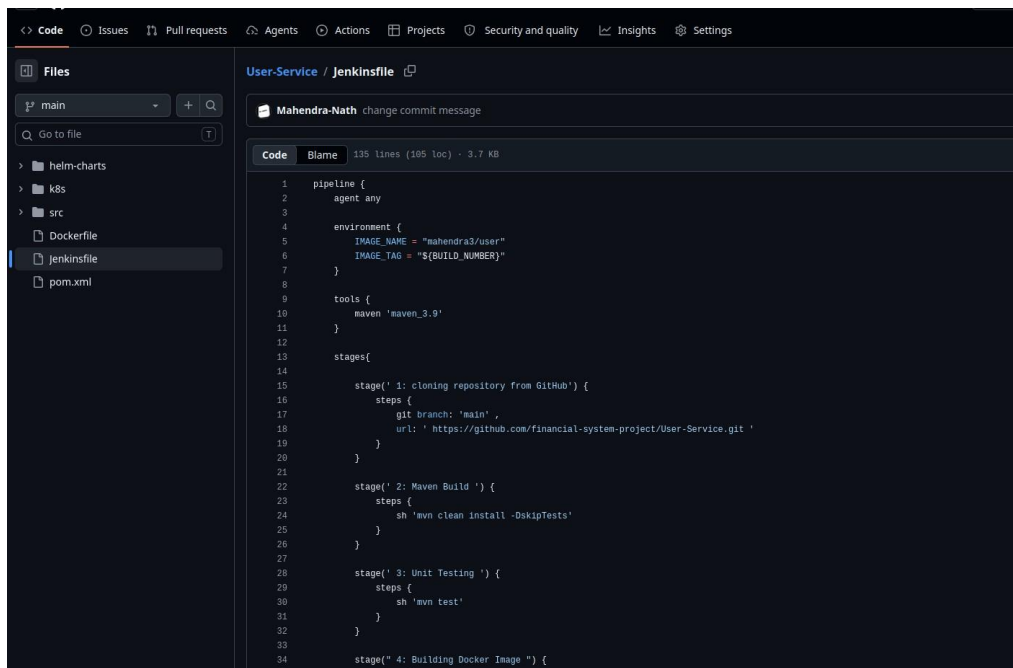


Figure 4: Jenkinsfile in the User-Service repository — pipeline stages defined as code

The eight stages declared in the Jenkinsfile are: *Clone from GitHub*, *Maven Build*, *Unit Testing*, *Build Docker Image*, *Push to DockerHub*, *Clean Docker Images*, *Update GitOps Repo*, and *Ansible Deployment*. The image tag used throughout the pipeline is the Jenkins build number (`${BUILD_NUMBER}`), which guarantees every image is uniquely versioned and traceable to a specific build.

3.3. GitOps Repository — Helm Charts and ArgoCD Mani-fests

The GitOps repository is the single source of truth for the cluster's desired state. It contains Helm charts for each service under `charts/`, and ArgoCD application manifests under `argocd-apps/`. Figure 5 shows the repository's `charts/` directory with folders for all five services and the ELK stack.

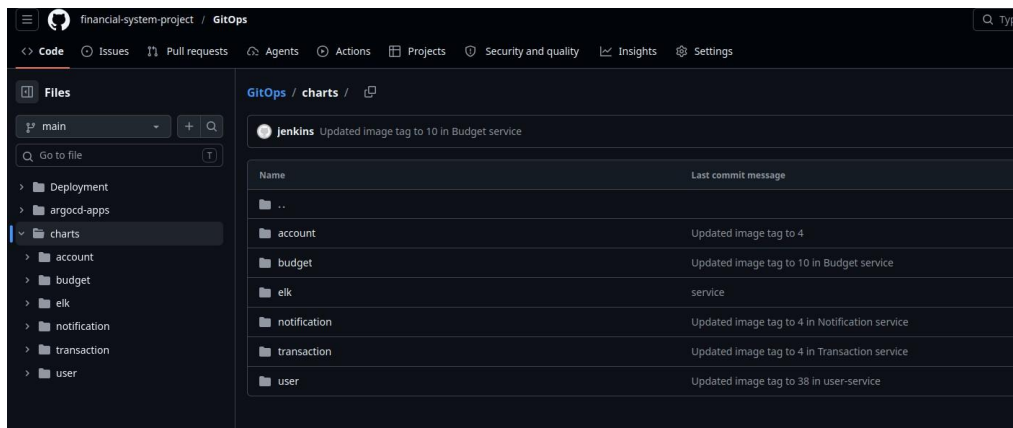


Figure 5: GitOps repository — charts/ directory with Helm charts for all microservices

A key detail visible in the commit messages is that the last write to each chart came from the Jenkins automation bot (e.g., *"Updated image tag to 10 in Budget service"*, *"Updated image tag to 38 in user-service"*). This confirms the CI-to-CD handshake is working correctly: Jenkins builds the image, pushes it to DockerHub, and then immediately updates the chart to reference the new tag.

3.4. Helm values.yaml — Image Tag Tracking

Within each service's Helm chart, the values.yaml file is the file that Jenkins updates on every successful build. Figure 6 shows the values.yaml for the User Service, where the image tag is set to 38, matching the latest Jenkins build. The file also contains the Vault server address and secret path, the Kubernetes namespace (bank-app-dev), resource limits, and health probe configuration — all managed as code.

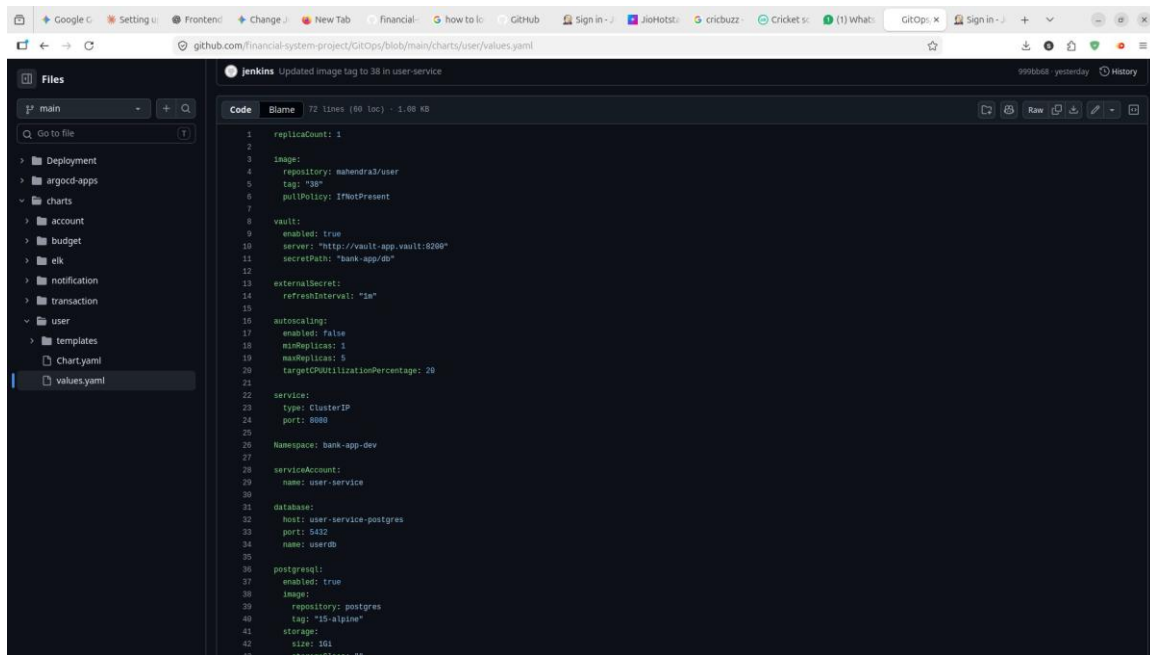


Figure 6: values.yaml for User Service — image tag updated to build #38 by Jenkins

4. Continuous Integration — Jenkins

4.1. Overview

Jenkins serves as the CI engine for the project. It is configured with a webhook integration so that every push to any service repository's main branch automatically triggers the corresponding pipeline. The pipelines are entirely declarative, driven by the Jenkinsfile in each repository.

4.2. All Pipelines — Dashboard View

Figure 7 shows the Jenkins dashboard with all five microservice pipelines. Every pipeline shows a green status on its last successful build, confirming that all five services are building and deploying successfully. The build history also reveals that the system has been iterated upon — the User Service pipeline has completed 38 builds, confirming active development.

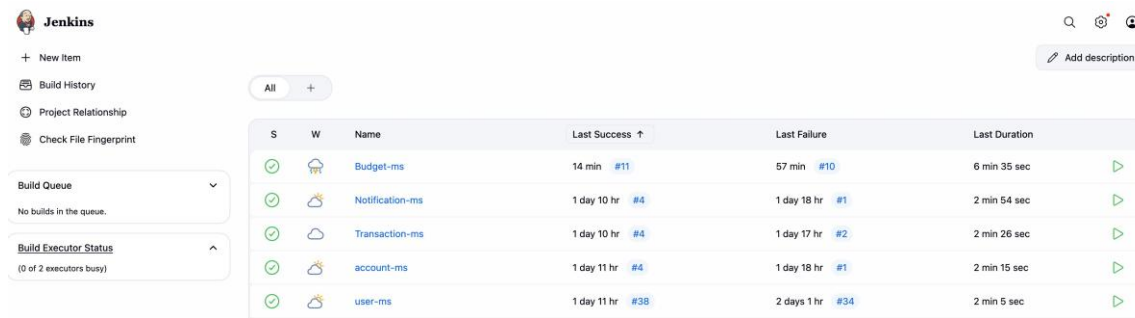


Figure 7: Jenkins dashboard — all five microservice pipelines with green build status

4.3. Stage View — User Service Pipeline

Figure 8 shows the Stage View for the user-ms pipeline. Each column represents one of the eight pipeline stages, and each row is a build. All stages for builds #36, #37, and #38 are green, with the full run completing in approximately two and a half minutes. The stage timing breakdown shows that pushing the Docker image to DockerHub is the longest step (around 35–43 seconds), while the ArgoCD sync via Ansible completes in about 50 seconds.

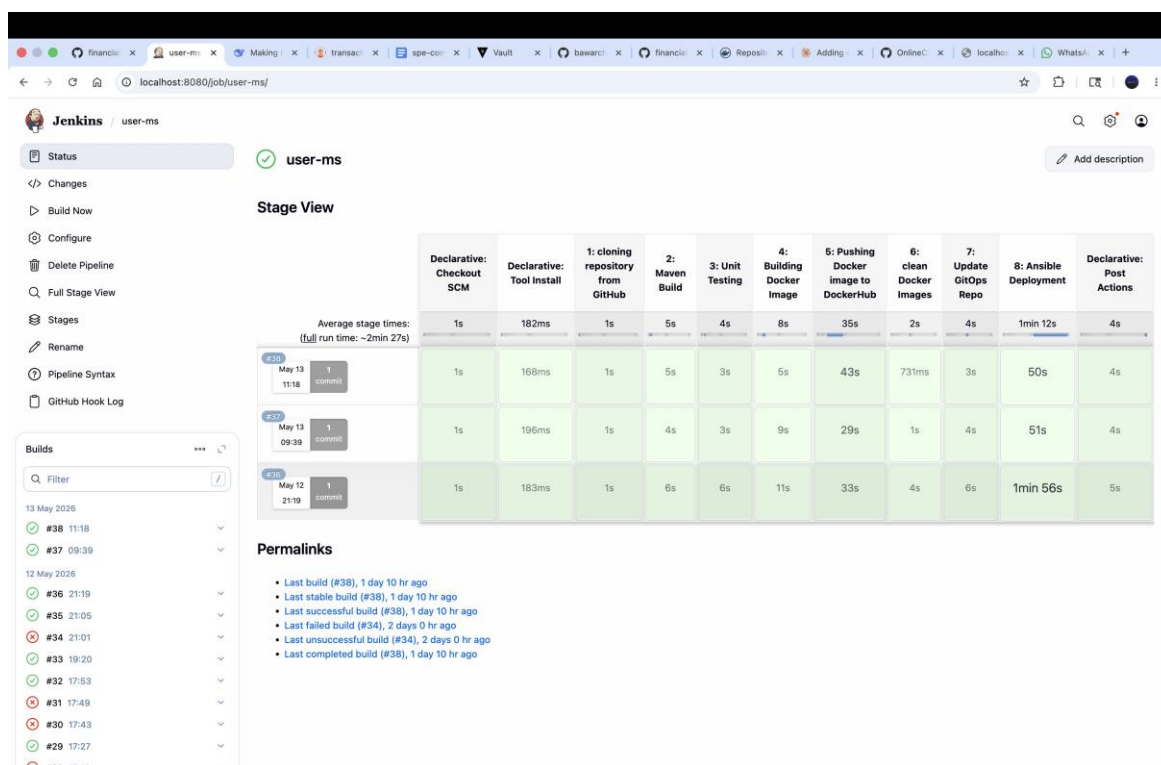
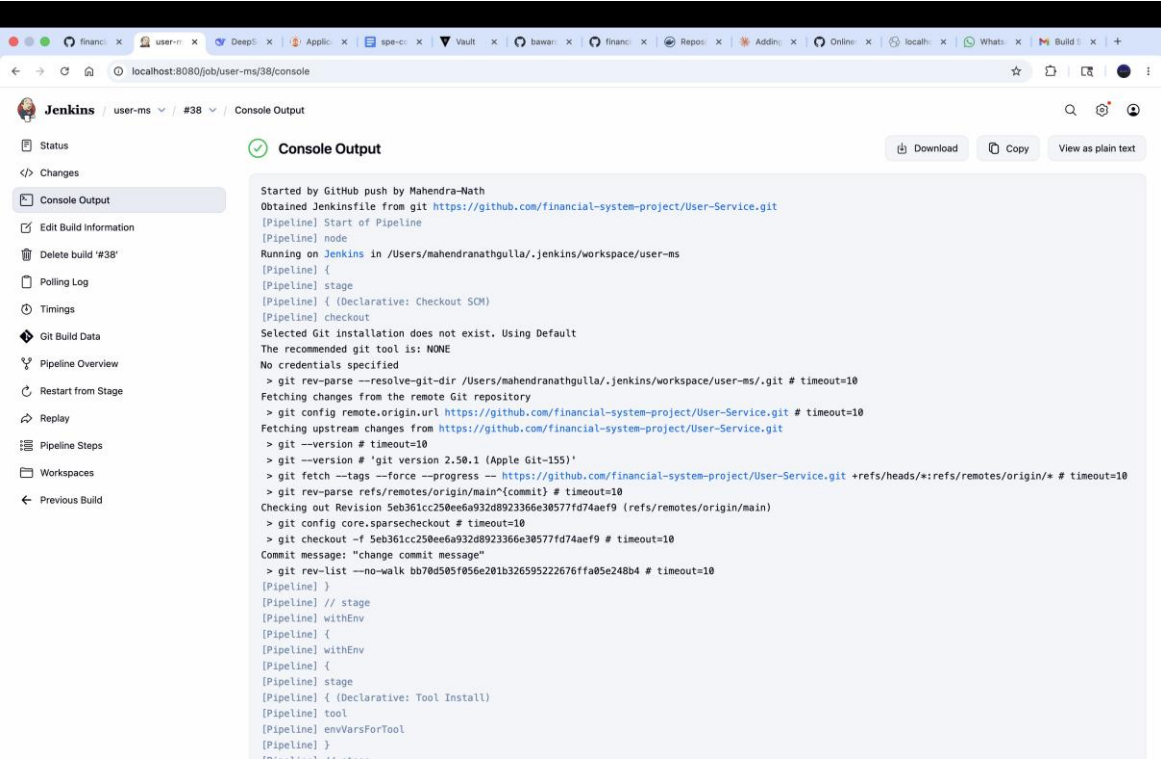


Figure 8: Jenkins Stage View for user-ms — all eight stages passing for builds #36–#38

4.4. Console Output — Full Build Trace

The Jenkins console output provides a complete audit trail of every build. The two screenshots below show the beginning and end of build #38 for the User Service.

Figure 9 shows the pipeline’s opening stages: Jenkins clones the repository from financial-system-project/User-Service.git on GitHub, identifying the commit that triggered the build.



```

Started by GitHub push by Mahendra-Nath
Obtained Jenkinsfile from git https://github.com/financial-system-project/User-Service.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /Users/mahendranathgulla/.jenkins/workspace/user-ms
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /Users/mahendranathgulla/.jenkins/workspace/user-ms/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/financial-system-project/User-Service.git # timeout=10
Fetching upstream changes from https://github.com/financial-system-project/User-Service.git
> git --version # timeout=10
> git --version # 'git version 2.50.1 (Apple Git-155)'
> git fetch --tags --force --progress -- https://github.com/financial-system-project/User-Service.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 5eb361cc250ee6a932d8923366e38577fd74aef9 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 5eb361cc250ee6a932d8923366e38577fd74aef9 # timeout=10
Commit message: "change commit message"
> git rev-list --no-walk bb78d505f056e201b326595222676ffa05e248b4 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Tool Install)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] }

```

Figure 9: Console output — pipeline start, Git checkout from GitHub (build #38)

Figure 10 shows the pipeline’s final stages: the Ansible playbook execution for the Ar-goCD sync. The output confirms that the ClusterSecretStore (vault-backend-user) is *Synced* and *Healthy*, the deployment commit is printed (*Author: jenkins — Com-mit: 999bb68 — Message: Updated image tag to 38 in user-service*), and the Ansible play recap shows *ok=8, failed=0*. The pipeline ends with **Finished: SUCCESS**.

```

Jenkins / user-ms / #38 / Console Output

"external-secrets.io ClusterSecretStore bank-app-dev vault-backend-user Running Synced clustersecretstore.external-
secrets.io/vault-backend-user unchanged",
"external-secrets.io ClusterSecretStore vault-backend-user Synced Healthy "
}
}

TASK [argocd : Get latest Git commit info] *****
ok: [localhost]

TASK [argocd : Print deployment commit details] *****
ok: [localhost] => {
  "msg": "Author: jenkins | Commit: 999bb68 | Message: Updated image tag to 38 in user-service"
}

PLAY RECAP *****
localhost : ok=8 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0

[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] mail
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

Figure 10: Console output — Ansible ArgoCD sync complete, Finished: SUCCESS (build #38)

4.5. Unit Testing — Maven and JUnit

Every pipeline runs `mvn test` as a mandatory gate before the Docker image is built. Figure 11 shows the test execution output from the User Service (build #38). Spring Boot starts the application context (`UserControllerTest` using Java 21.0.1), runs six test cases, and reports **Tests run: 6, Failures: 0, Errors: 0, Skipped: 0**, followed by **BUILD SUCCESS**. If any test fails, the pipeline aborts at this stage and does not produce a Docker image, preventing broken code from reaching the cluster.

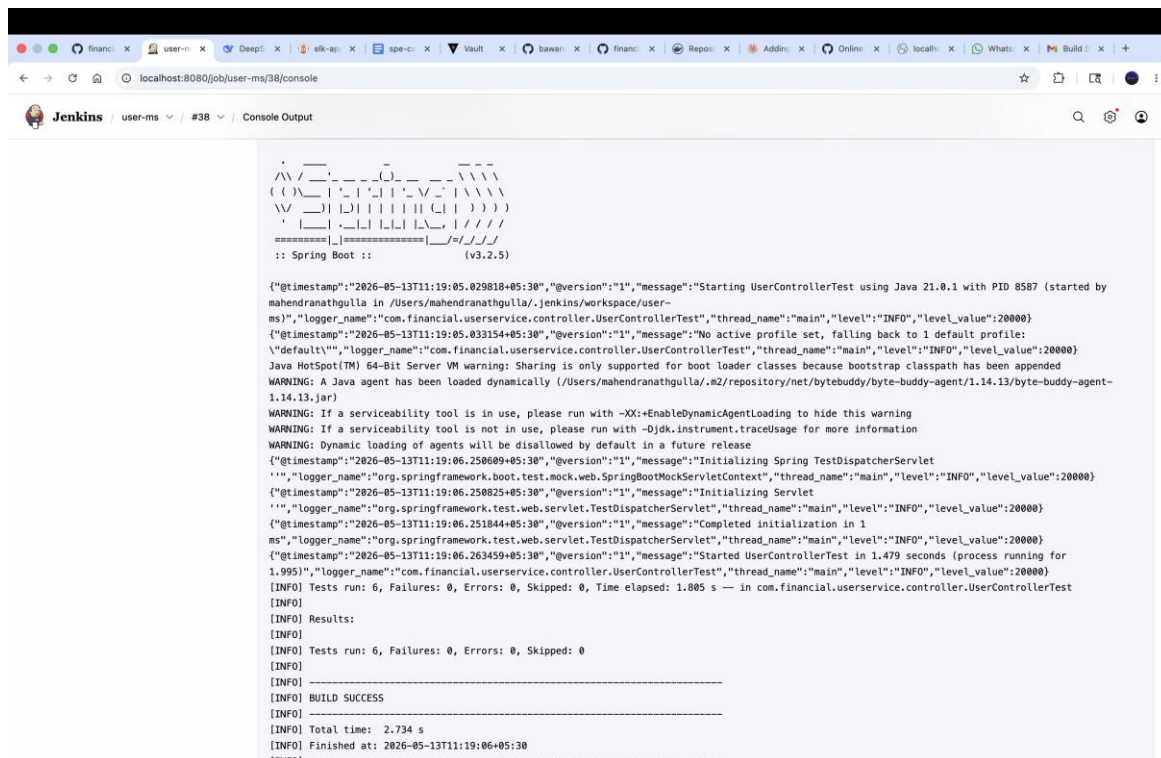


Figure 11: JUnit test results in Jenkins console — 6 tests passed, BUILD SUCCESS

4.6. Jenkins Credentials — Secure Secret Management

Sensitive values such as DockerHub credentials, the GitHub Personal Access Token, and ArgoCD login credentials are stored in Jenkins' encrypted credential store and never appear in plain text in logs or Jenkinsfiles. Figure 12 shows the credentials page with all secrets registered: DockerPwd (DockerHub), github-token (GitHub PAT), argocd-creds (ArgoCD login), and a webhook secret. Jenkinsfiles reference these by their ID using the `withCredentials` block, so even if a Jenkinsfile is public, no secrets are exposed.

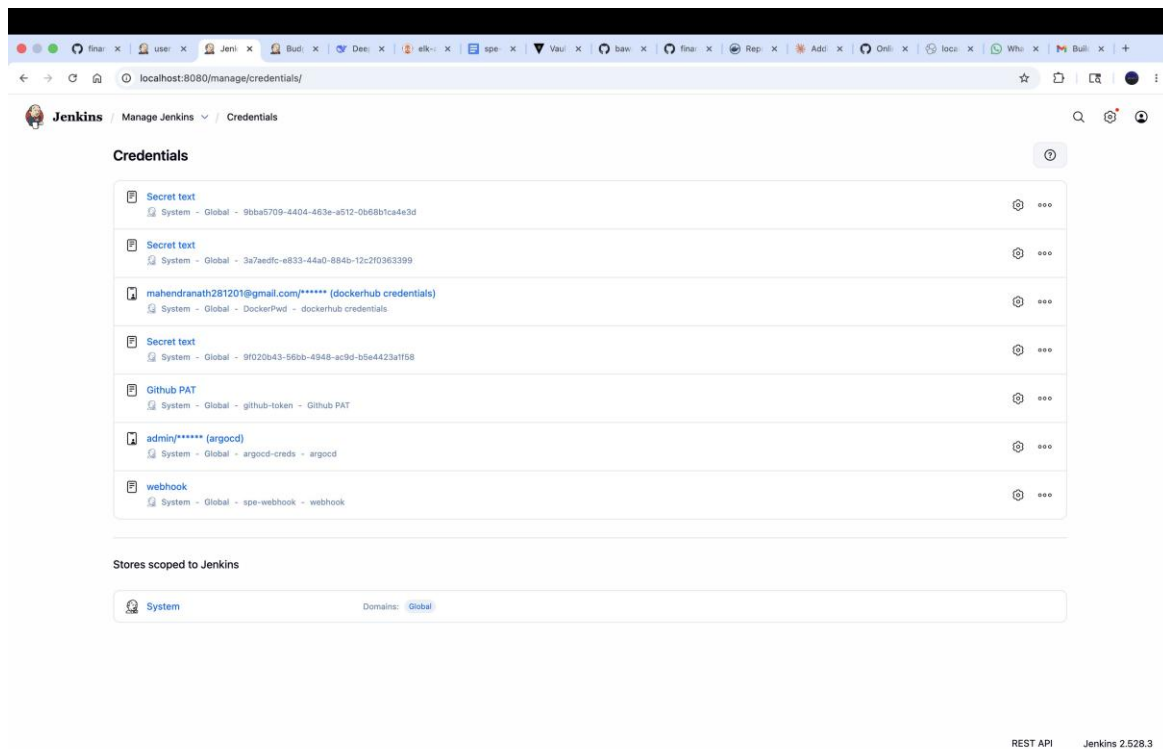


Figure 12: Jenkins Credentials store — DockerHub, GitHub PAT, ArgoCD, and web-hook secrets

5. Container Registry — DockerHub

5.1. Image Repositories

Every microservice has a dedicated DockerHub image repository under the mahendra3 namespace. Figure 13 shows all five service image repositories: mahendra3/notification, mahendra3/budget, mahendra3/transaction, mahendra3/account, and mahendra3/user. All were last pushed one day ago, confirming recent successful builds across the board. A Helm chart repository (mahendra3/user-service) is also visible.

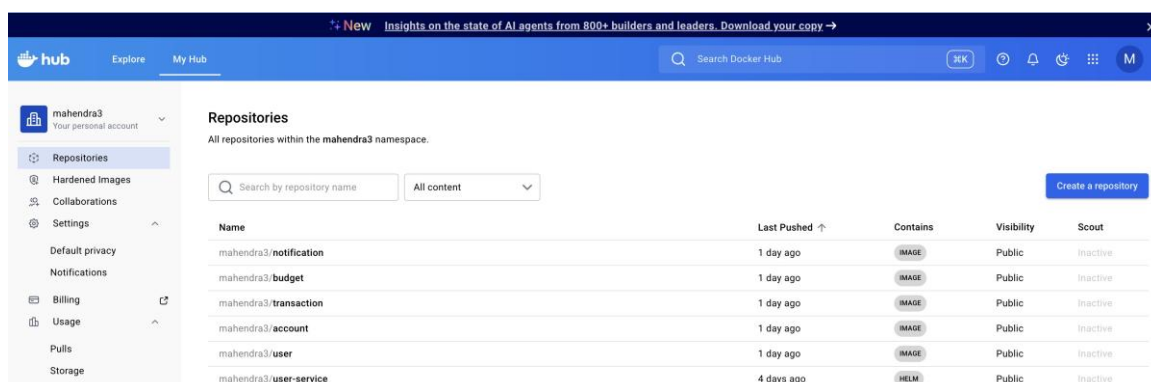


Figure 13: DockerHub repositories — all five microservice images under the mahendra3 namespace

5.2. Versioned Image Tags

Figure 14 shows the tags for the mahendra3/user image. Tags 36, 37, 38, and latest are present, directly corresponding to Jenkins build numbers. The repository size of 1.3 GB and 213 total pulls indicate active usage. This versioning strategy means that any previous deployment can be rolled back simply by updating the image.tag in the values.yaml to an earlier build number — ArgoCD will automatically reconcile the change.

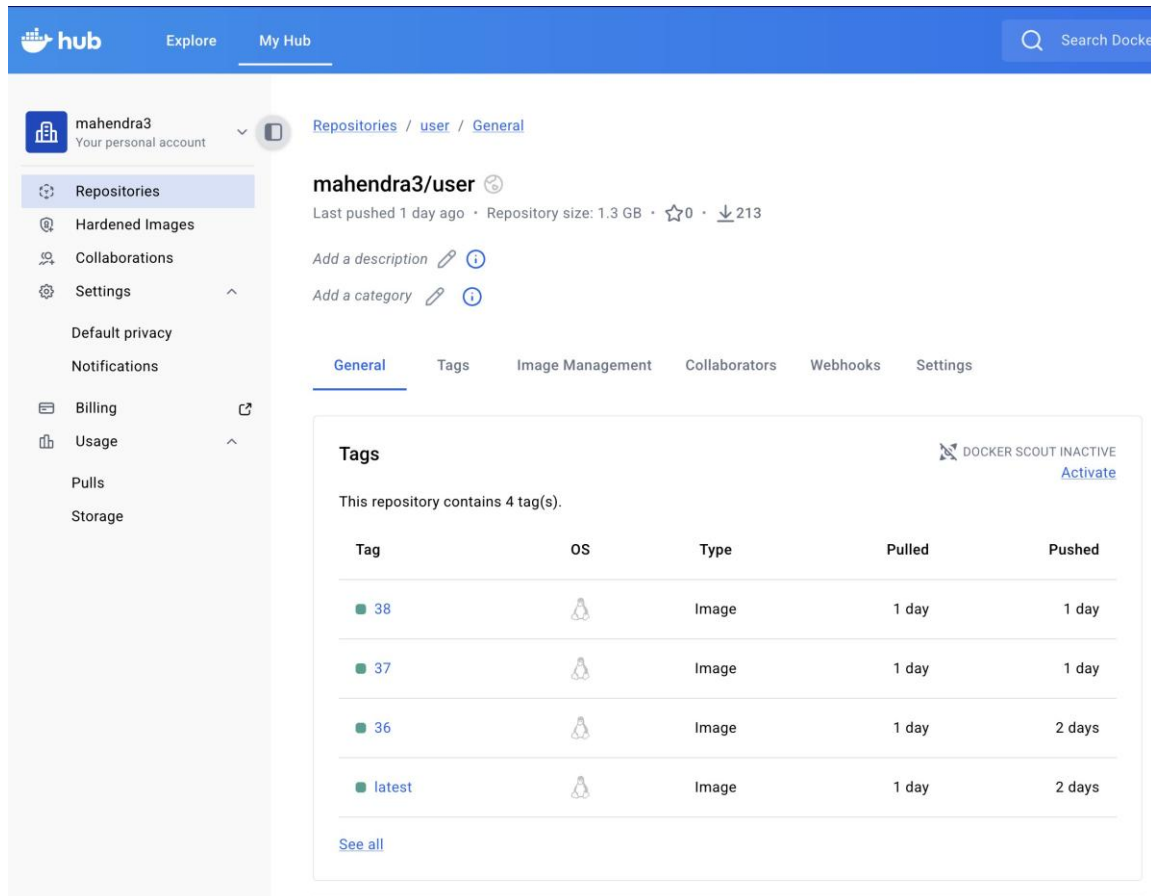


Figure 14: mahendra3/user image tags — builds #36, #37, #38 with latest, matching Jenkins build numbers

6. Secrets Management — HashiCorp Vault

6.1. Overview

A critical requirement for any production financial system is that database credentials, API keys, and other sensitive configuration values must never be hardcoded in source code or Kubernetes manifests. This project uses **HashiCorp Vault** (KV v2 secrets engine) together with the **External Secrets Operator** to inject credentials securely into pods at runtime.

The integration works as follows: Vault stores credentials at specific paths. A ClusterSecretStore resource in Kubernetes authenticates to Vault using the Kubernetes auth method (a service account token). An ExternalSecret resource specifies which Vault path to read and creates a native Kubernetes Secret automatically. The Helm chart's Deployment then mounts this Secret as environment variables, so the application receives DB_USER and DB_PASSWORD without the values ever appearing in any manifest or source file.

6.2. Vault Secrets — Per-Service Paths

Figure 15 shows the Vault UI under the bank-app/ prefix in the secret KV v2 engine. Separate secret paths exist for each service: db, db-account, db-budget, db-notification, and db-transaction. This per-service isolation means a compromise of one service's credentials does not expose other services.

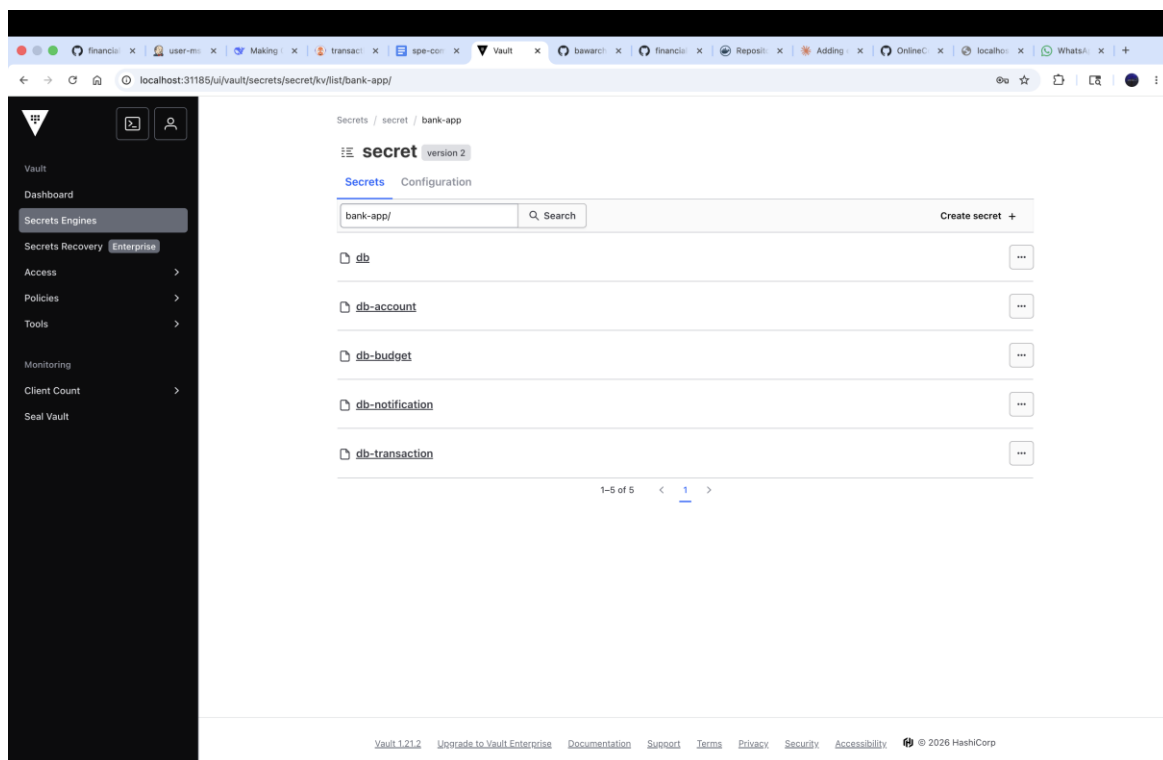


Figure 15: HashiCorp Vault — bank-app/ secret paths for each microservice database

6.3. Secret Key-Value Contents

Figure 16 shows the contents of bank-app/db-account. Two keys are stored — username and password — with their values masked in the UI. This is the exact path referenced in the Account Service's Helm chart values.yaml (vault.secretPath: "bank-app/db-account"), and the ExternalSecret maps these keys to the DB_USER and DB_PASSWORD environment variables consumed by the Account Service pod.

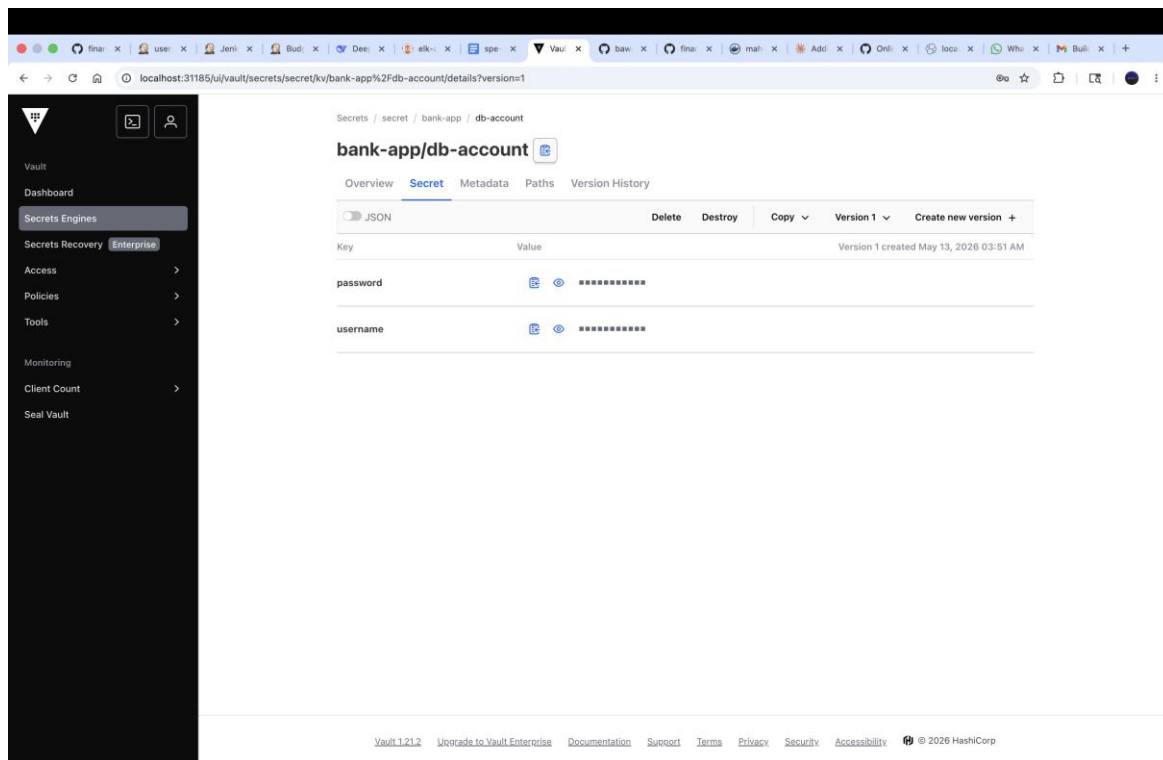


Figure 16: Vault secret at bank-app/db-account — username and password keys, values masked

6.4. Kubernetes Authentication — bank-app-role

Figure 17 shows the bank-app-role configured under the Vault Kubernetes auth method. This role is bound to the external-secrets service account in the external-secrets namespace. When the External Secrets Operator pod (running as that service account) requests a token from Vault, Vault verifies the Kubernetes service account JWT, confirms it matches this role, and issues a short-lived Vault token. The policy attached to this role (user-service-static) grants read-only access to the relevant secret paths — following the principle of least privilege.

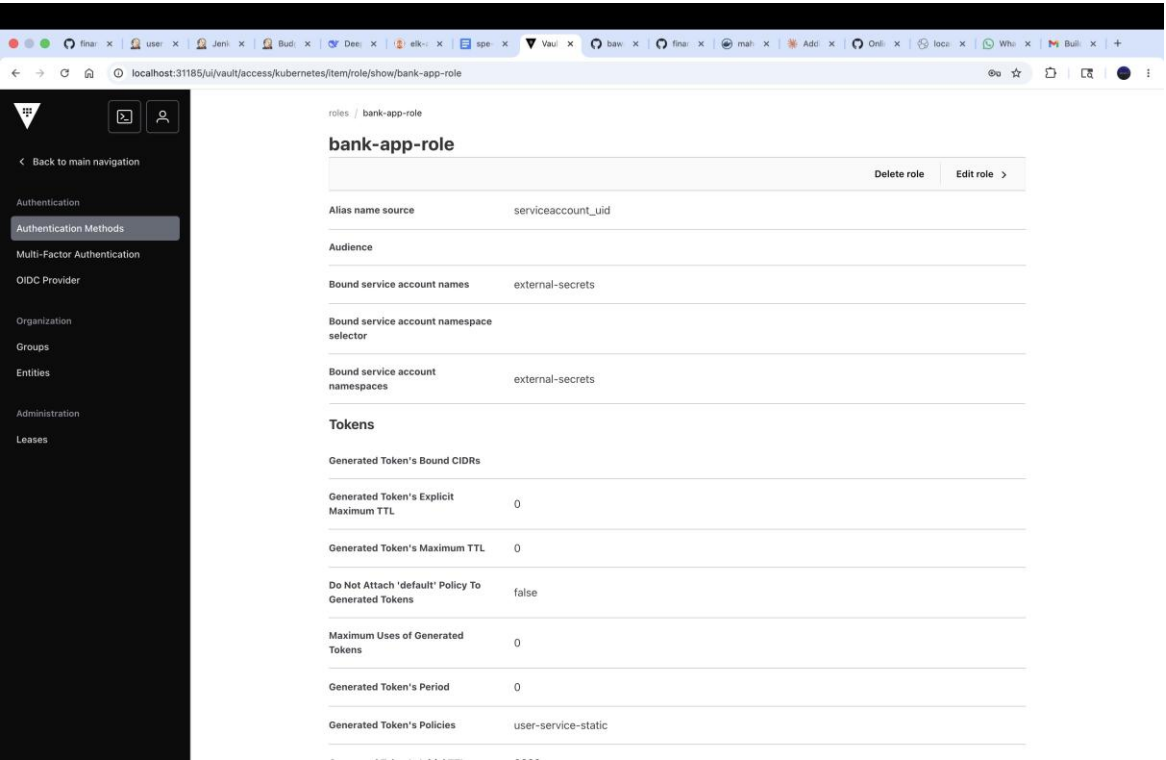


Figure 17: Vault Kubernetes auth — bank-app-role bound to the external-secrets service account

Section 7: ArgoCD — GitOps Continuous Delivery

ArgoCD serves as the GitOps-based continuous delivery engine for the entire system. It watches the financial-system-project/GitOps GitHub repository and automatically reconciles the live Kubernetes cluster state with whatever is declared in that repo. The deployment model used here is the **App-of-Apps pattern**: a single root application called main-app manages all child applications, making it possible to bootstrap every microservice deployment with a single `kubectl apply`.

App-of-Apps: the main-app

The main-app ArgoCD Application points to the `argocd-apps/microservice-apps/` folder in the GitOps repo. When ArgoCD syncs this path it discovers and instantiates all the child application manifests in that directory — `account-app`, `budget-app`, `elk-app`, `notification-app`, `transaction-app`, and `user-app` — without any manual intervention. The screenshot below shows the main-app tree view after a Jenkins-triggered sync. The **App Health** reads **Healthy**, **Sync Status** reads **Synced** to HEAD (`cba696b`), and the **Last Sync OK** confirms the most recent successful reconciliation. The commit comment visible in the panel — *“Updated image tag to 11 in Budget service”* — proves that this sync was triggered automatically by the Jenkins pipeline after a successful Docker image push.

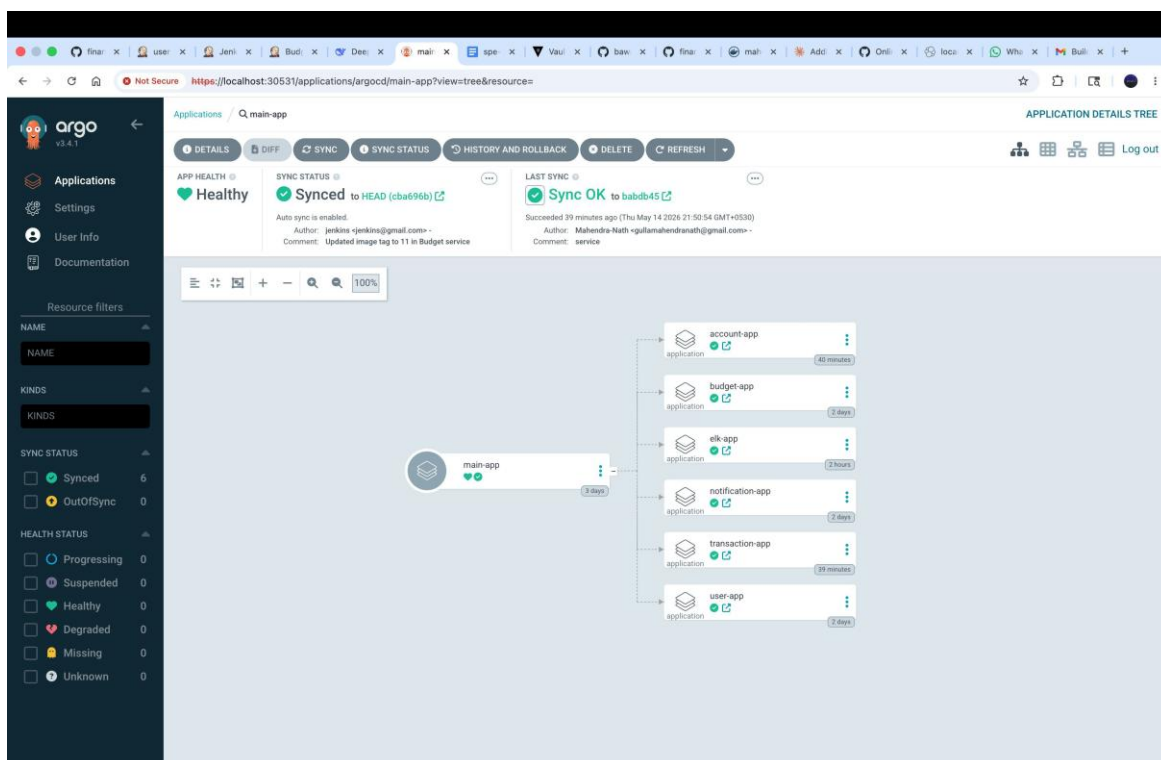


Figure 18: ArgoCD App-of-Apps: main-app tree view showing all six child applications (ac-count, budget, elk, notification, transaction, user) all in a Synced and Healthy state.

All Applications — Tile View

The ArgoCD Applications tile view gives a consolidated status for every application managed by the system. All six applications — `account-app`, `budget-app`, `main-app`, `notification-app`, `transaction-app`, and `user-app` — show **Healthy** and **Synced** simultaneously, all sourced from <https://github.com/financial-system-project/GitOps.git> at revision `HEAD`. The Helm chart paths for each service (`charts/account`, `charts/budget`, etc.) and the destination namespace `bank-app-dev` are clearly visible for each tile. The `main-app` tile shows its path as `./argocd-apps/microservice-apps`, confirming the App-of-Apps structure. Six out of six applications report **Sync OK**, and the sync timestamps are all within minutes of each other, demonstrating coordinated, automated deployment.

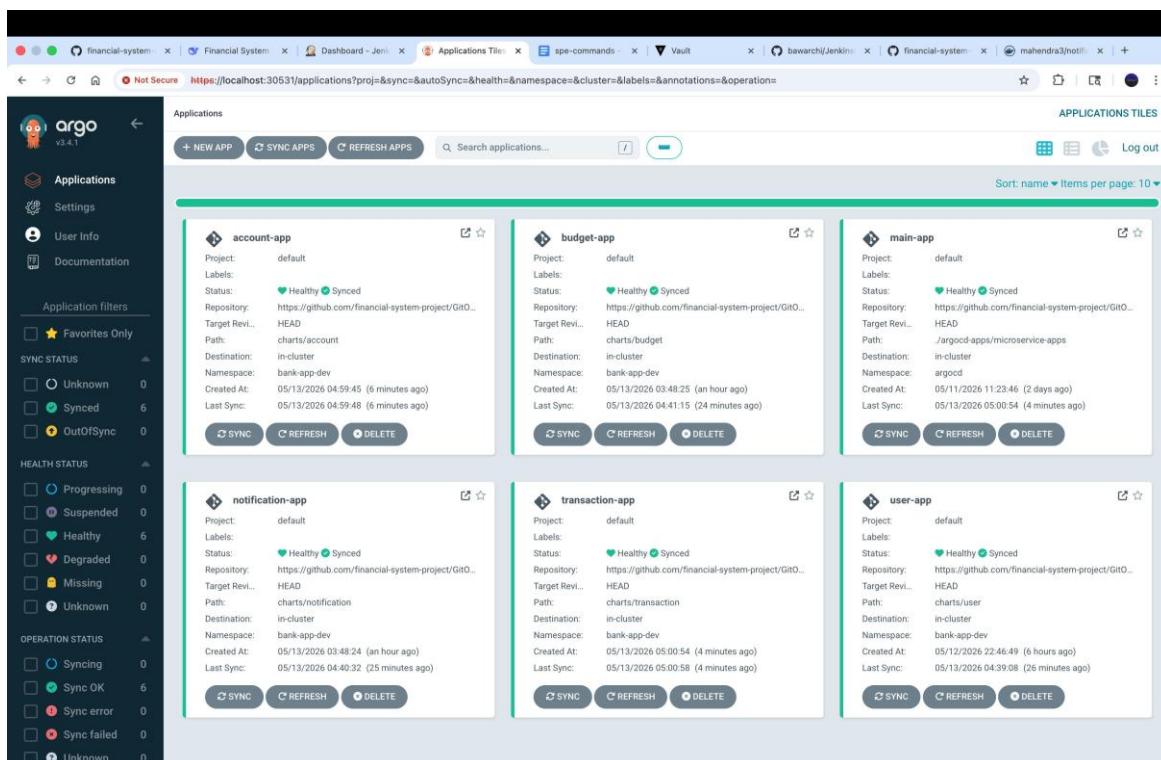


Figure 19: ArgoCD Applications tile view: all six applications are Healthy and Synced, all pointing to the same GitOps repository with their respective Helm chart paths.

Application Resource Tree — `account-app`

Clicking into the `account-app` reveals the full Kubernetes resource tree that ArgoCD rendered from the Helm chart. Every resource in the chart is present and healthy: the **Con-figMap** (`account-service-config`), the application **Service** (`account-service`), the PostgreSQL headless **Service** (`account-service-postgres`), the application **Deployment** which has rolled out to a `ReplicaSet` and a running `Pod`, the PostgreSQL **State-fulSet** with its own `pod` (`account-service-postgres-0`) and a bound `PersistentVolume-Claim`, the **ClusterSecretStore** (`vault-backend-account`) that connects to HashiCorp Vault using Kubernetes authentication, and the **ExternalSecret** (`account-service-secret`) that fetches database credentials from Vault and materialises them as a Kubernetes `Secret`. All resources show the green health icon, and the `pod` is reported as running `1/1`.

This single screenshot demonstrates the entire stack — application, database, and secrets

management — deployed and healthy through GitOps automation.

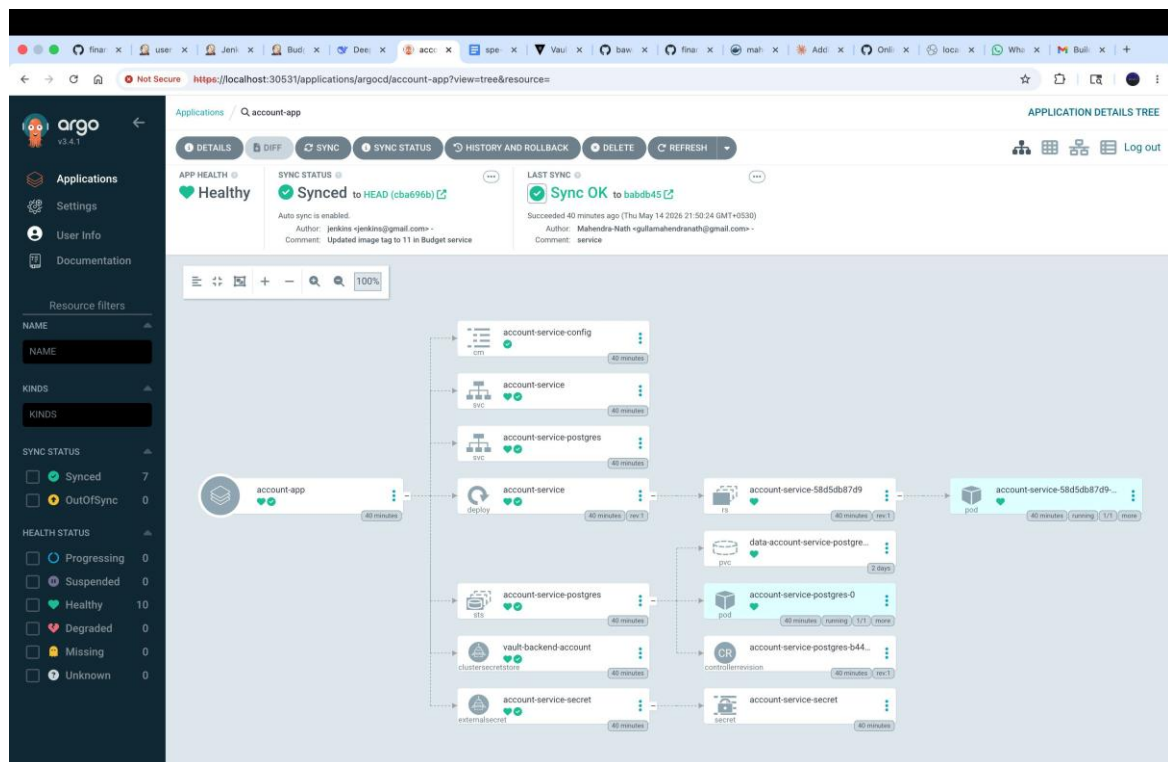


Figure 20: ArgoCD resource tree for account-app: ConfigMap, Services, Deployment, State-fulSet (PostgreSQL), PVC, ClusterSecretStore, ExternalSecret, and Kubernetes Secret all de-ployed and healthy.

Deployment History and GitOps Loop Proof

The History and Rollback panel for the user-app provides concrete evidence that the full GitOps loop — Jenkins CI commits a new image tag to the GitOps repository, and ArgoCD automatically detects and deploys it — is working end-to-end. The history shows multiple deployments, each authored by jenkins <jenkins@gmail.com> with commit messages of the form “Updated image tag to 38 in user-service” and “Updated image tag to 3”. The deploy time, time-to-deploy (as fast as one second for Helm-templated manifests), and the initiating actor (admin via the ArgoCD API triggered by Ansible) are all recorded. This rollback history also means any deployment can be reverted to a previous image tag with a single click, providing full auditability and disaster recovery without touching any YAML files manually.

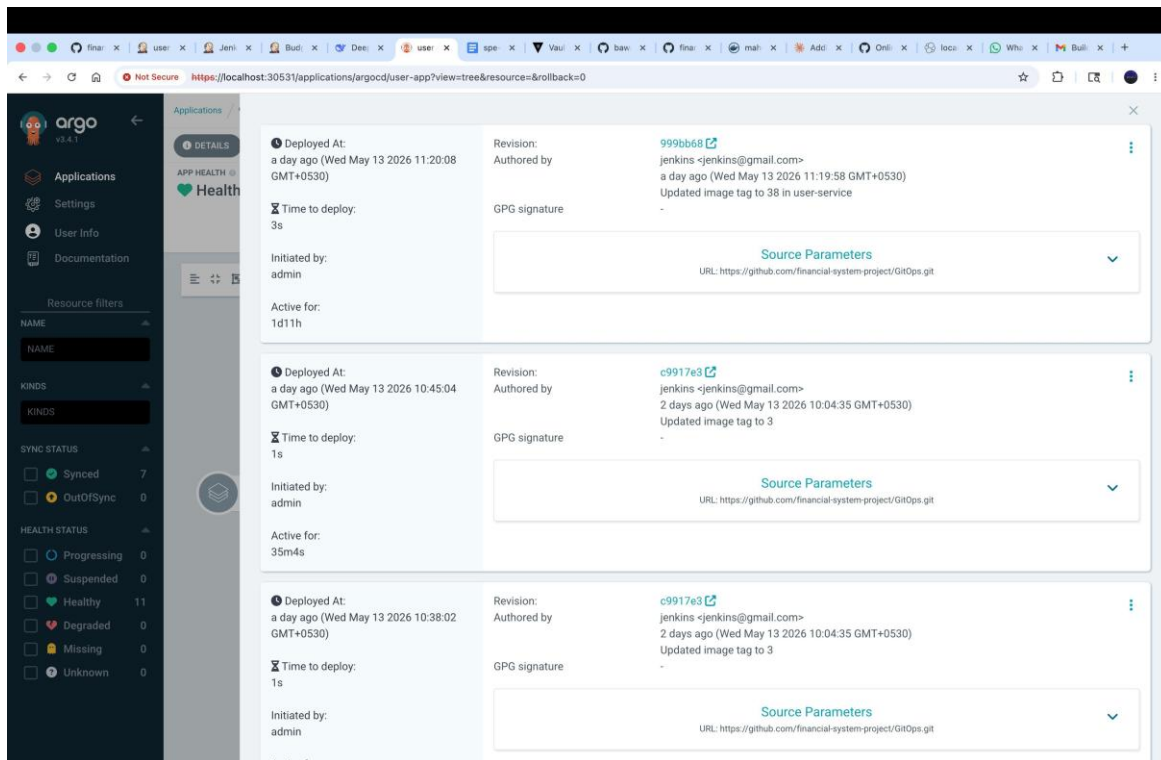


Figure 21: ArgoCD deployment history for user-app: multiple Jenkins-authored commits with messages confirming automated image tag updates, deployment timestamps, and deploy durations.

Section 8: Kubernetes Cluster

All microservices and their databases are deployed into the bank-app-dev Kubernetes namespace. ArgoCD uses the Helm charts in the GitOps repository to render every manifest — Deployments, StatefulSets, Services, ConfigMaps, ExternalSecrets — and applies them to the cluster. The following screenshots validate the live cluster state.

Running Pods in bank-app-dev

Running `kubectl get pods -n bank-app-dev` shows ten pods, all in Running status with 1/1 containers ready. Each of the five microservices has exactly one application pod and one PostgreSQL StatefulSet pod:

- account-service-58d5db87d9-snj9n and account-service-postgres-0
- budget-service-64c97b8cdf-rc5pj and budget-service-postgres-0
- notification-service-5fc5548bc6-rnskh and notification-service-postgres-0
- transaction-service-9b97c7789-bncn4 and transaction-service-postgres-0
- user-service-b958886cf-nh212 and user-service-postgres-0

The pod ages confirm that the PostgreSQL StatefulSets have been running stably (up to 47 hours in the case of user-service-postgres-0), while the application pods are more recent due to rolling updates from new Jenkins builds.

```
(base) mahendranathgulla@Mahendranaths-MacBook-Air ~ % kubectl get pods -n bank-app-dev
```

NAME	READY	STATUS	RESTARTS	AGE
account-service-58d5db87d9-snj9n	1/1	Running	11	43m
account-service-postgres-0	1/1	Running	1	43m
budget-service-64c97b8cdf-rc5pj	1/1	Running	0	8m26s
budget-service-postgres-0	1/1	Running	6	42h
notification-service-5fc5548bc6-rnskh	1/1	Running	15	34h
notification-service-postgres-0	1/1	Running	6	42h
transaction-service-9b97c7789-bncn4	1/1	Running	11	43m
transaction-service-postgres-0	1/1	Running	1	43m
user-service-b958886cf-nh2l2	1/1	Running	3	23m
user-service-postgres-0	1/1	Running	6	47h

```
(base) mahendranathgulla@Mahendranaths-MacBook-Air ~ %
```

Figure 22: kubectl get pods -n bank-app-dev: all ten pods (five application pods and five PostgreSQL pods) in Running status with all containers ready.

Services and Cluster Networking

Running kubectl get svc -n bank-app-dev lists all ten Kubernetes Services. Each microservice exposes a ClusterIP service on its designated port (User: 8080, Account: 8081, Transaction: 8082, Budget: 8083, Notification: 8084), enabling intra-cluster HTTP communication between services via stable DNS names. Each PostgreSQL instance has a headless service (ClusterIP: None) on port 5432, which is the standard Kubernetes pattern for StatefulSet stable network identity — the application connects to <service-name> and DNS resolves to the pod's IP directly.

```
(base) mahendranathgulla@Mahendranaths-MacBook-Air ~ % kubectl get svc -n bank-app-dev
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
account-service	ClusterIP	10.104.8.206	<none>	8081/TCP	47m
account-service-postgres	ClusterIP	None	<none>	5432/TCP	47m
budget-service	ClusterIP	10.107.136.145	<none>	8083/TCP	42h
budget-service-postgres	ClusterIP	None	<none>	5432/TCP	42h
notification-service	ClusterIP	10.97.114.185	<none>	8084/TCP	42h
notification-service-postgres	ClusterIP	None	<none>	5432/TCP	42h
transaction-service	ClusterIP	10.105.134.42	<none>	8082/TCP	46m
transaction-service-postgres	ClusterIP	None	<none>	5432/TCP	46m
user-service	ClusterIP	10.105.228.48	<none>	8080/TCP	47h
user-service-postgres	ClusterIP	None	<none>	5432/TCP	47h

```
(base) mahendranathgulla@Mahendranaths-MacBook-Air ~ %
```

Figure 23: kubectl get svc -n bank-app-dev: five application ClusterIP services and five PostgreSQL headless services, all with correct port assignments.

ExternalSecrets Synced from HashiCorp Vault

This is one of the most important validations in the project. Running kubectl get externalsecret -n bank-app-dev shows all five ExternalSecret resources, each with **STATUS: SecretSynced** and **READY: True**. Each ExternalSecret references its re-spective ClusterSecretStore (e.g., vault-backend-account), which in turn authenticates to HashiCorp Vault using the cluster's Kubernetes service account token. Vault returns the DB.USER and DB.PASSWORD values stored at the secret/bank-app/db path, and the External Secrets Operator materialises these as a standard Kubernetes Secret. The refresh interval is set to **1 minute**, meaning that if a database password is rotated

in Vault, all pods automatically receive the updated credential within 60 seconds. No secrets are stored in the GitOps repository or in any application configuration file.

```
((base) mahendranathgulla@Mahendranaths-MacBook-Air ~ % kubectl get externalsecret -n bank-app-dev
NAME                                STORETYPE      STORE              REFRESH INTERVAL  STATUS      READY  LAST SYNC
account-service-secret             ClusterSecretStore vault-backend-account 1m      SecretSynced  True   3s
budget-service-secret              ClusterSecretStore vault-backend-budget  1m      SecretSynced  True   3s
notification-service-secret        ClusterSecretStore vault-backend-notification 1m      SecretSynced  True   1s
transaction-service-secret         ClusterSecretStore vault-backend-transaction 1m      SecretSynced  True   9s
user-service-secret                ClusterSecretStore vault-backend-user      1m      SecretSynced  True   7s
(base) mahendranathgulla@Mahendranaths-MacBook-Air ~ %
```

Figure 24: `kubectl get externalsecret -n bank-app-dev`: all five ExternalSecrets show SecretSynced / Ready: True, proving live Vault integration via the External Secrets Operator with a 1-minute refresh cycle.

ArgoCD Application Status via CLI

Running `argocd app list` from the terminal provides a machine-readable confirmation of what the ArgoCD UI shows. All seven managed applications — `account-app`, `budget-app`, `elk-app`, `main-app`, `notification-app`, `transaction-app`, and `user-app` — show **STATUS: Synced**, **HEALTH: Healthy**, and **SYNCPOLICY: Auto-Prune**. The CLI output also confirms each application’s repository URL, Helm chart path, and target revision (HEAD), providing an unambiguous audit trail of the deployment configuration. This output is useful for scripting health checks and integrating ArgoCD status into wider monitoring pipelines.

```
((base) mahendranathgulla@Mahendranaths-MacBook-Air ~ % argocd app list
NAME      CLUSTER      NAMESPACE      PROJECT  STATUS  HEALTH  SYNCPOLICY  CONDITIONS  REPO      PATH      TARGET
argocd/account-app  https://kubernetes.default.svc  bank-app-dev  default  Synced  Healthy  Auto-Prune  <none>      https://github.com/financial-system-project/GitOps.git  charts/account  HEAD
argocd/budget-app   https://kubernetes.default.svc  bank-app-dev  default  Synced  Healthy  Auto-Prune  <none>      https://github.com/financial-system-project/GitOps.git  charts/budget   HEAD
argocd/elk-app      https://kubernetes.default.svc  logging       default  Synced  Healthy  Auto-Prune  <none>      https://github.com/financial-system-project/GitOps.git  charts/elk      HEAD
argocd/main-app     https://kubernetes.default.svc  argocd        default  Synced  Healthy  Auto-Prune  <none>      https://github.com/financial-system-project/GitOps.git  ./argocd-apps/microservice-apps  HEAD
argocd/notification-app https://kubernetes.default.svc  bank-app-dev  default  Synced  Healthy  Auto-Prune  <none>      https://github.com/financial-system-project/GitOps.git  charts/notification  HEAD
argocd/transaction-app https://kubernetes.default.svc  bank-app-dev  default  Synced  Healthy  Auto-Prune  <none>      https://github.com/financial-system-project/GitOps.git  charts/transaction  HEAD
argocd/user-app     https://kubernetes.default.svc  bank-app-dev  default  Synced  Healthy  Auto-Prune  <none>      https://github.com/financial-system-project/GitOps.git  charts/user       HEAD
(base) mahendranathgulla@Mahendranaths-MacBook-Air ~ %
```

Figure 25: `argocd app list`: all seven applications (including `elk-app` in the logging namespace) are Synced, Healthy, and governed by Auto-Prune sync policy.

Section 9: API Testing — End-to-End Service Verification

With all services running in Kubernetes, the following tests were performed using Post-man against the port-forwarded service endpoints. The tests follow a realistic financial workflow: register a user, open accounts, fund them, perform a transfer, set a budget and track spending, and trigger a budget alert notification. Each step validates not only the service itself but also its connectivity to PostgreSQL (via Vault-injected credentials) and to other services via Feign clients.

User Service — Register User

A POST to `/api/users/register` with a JSON body containing username, email, password, and `fullName` returns **201 Created** with the persisted user object, including a database-generated id of 3. This confirms that the User Service is running, connected to its Post-greSQL database (credentials sourced from Vault), and able to write and return records.

The email (`john@test.com`) and other fields are echoed back exactly, confirming proper JPA persistence.

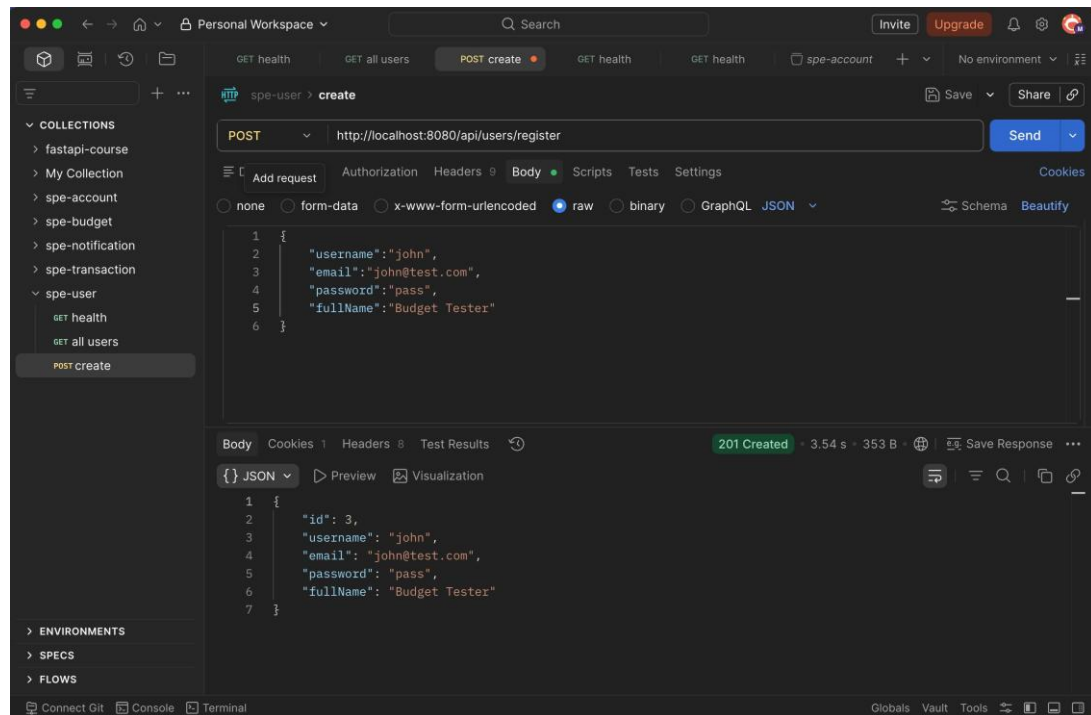


Figure 26: User Service: POST /api/users/register returns **201 Created** with the newly created user record including an auto-generated ID.

Account Service — Create Account

A POST to /api/accounts with `userId: 1` and `accountType: "CHECKING"` returns **201 Created**. The Account Service validates the existence of the referenced user by making a Feign HTTP call to the User Service (user-service:8080) before persisting the account. The response includes an auto-generated `accountNumber` (ACC-27d6e0f8), confirming UUID-based account numbering, and an initial balance of 0. This test proves both that the Account Service is connected to its own database and that inter-service communication between Account and User is working inside the cluster.

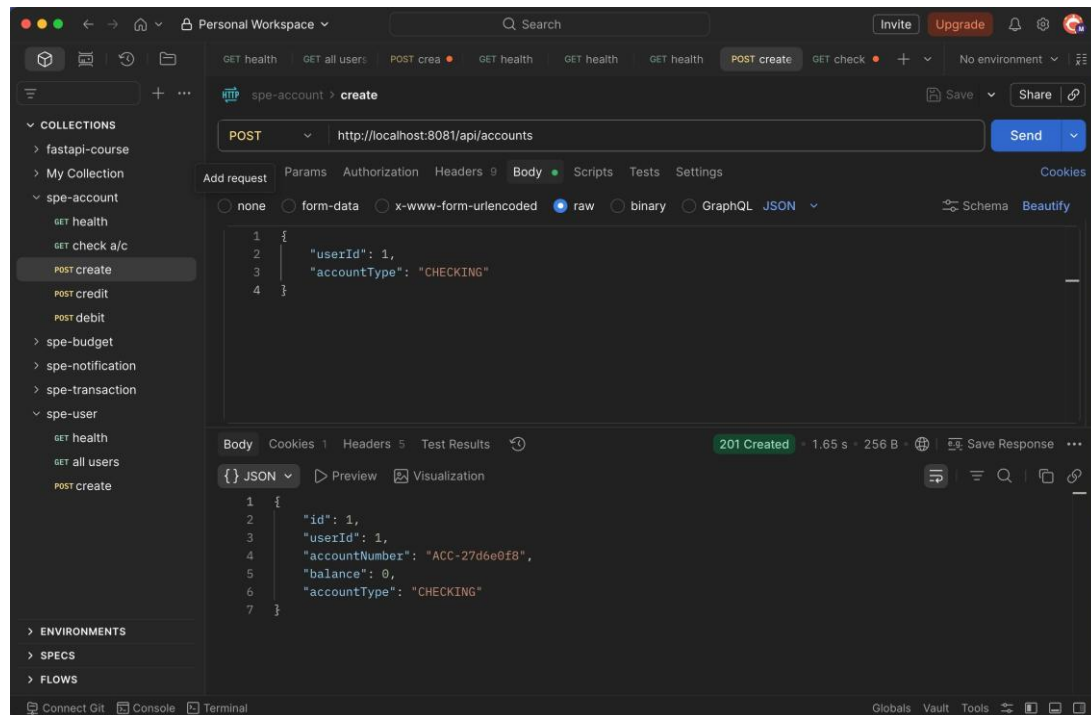
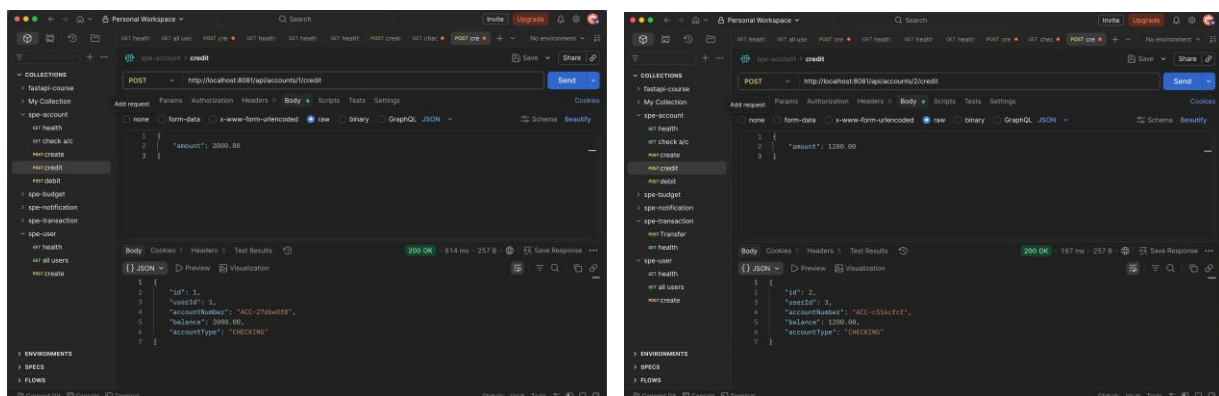


Figure 27: Account Service: POST /api/accounts returns **201 Created** with the new account details, including a UUID-based account number and zero initial balance.

Transaction Service — Fund Accounts and Transfer

To test the Transaction Service’s Saga-based transfer logic, two accounts were first funded via the Account Service’s credit endpoint. Account 1 (ACC-27d6e0f8) was credited with **\$2000.00** and Account 2 (ACC-c516cfcf) was credited with **\$1200.00**. Both credit operations returned **200 OK** with updated balances.



(a) Account 1 credited \$2000 (balance: 2000.00)

(b) Account 2 credited \$1200 (balance: 1200.00)

Figure 28: Pre-transfer funding: both accounts credited via the Account Service before initiating the transfer.

A POST to /api/transactions/transfer with fromAccountId: 1, toAccountId: 2, and amount: 1000.00 returned **200 OK** with status: "COMPLETED" and message: "Transfer successful". This result confirms the full Saga execution: the Transaction Service validated both accounts via Feign calls to the Account Service, debited Account

1, and credited Account 2 atomically. The transactionId: 1 is the database record created by the service, and the timestamp confirms live execution.

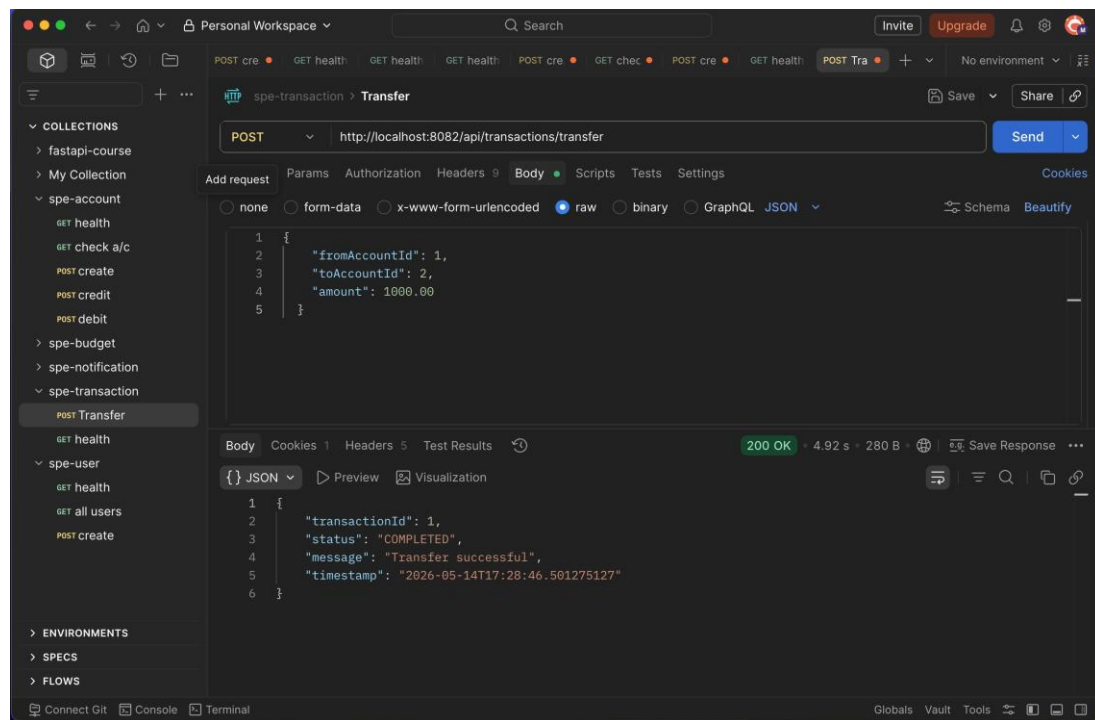


Figure 29: Transaction Service: POST `/api/transactions/transfer` returns **200 OK** with status **COMPLETED** and message “Transfer successful”, confirming the Saga Orchestrator completed debit and credit steps successfully.

To verify that the balances were actually updated, a GET on Account 2 (`/api/accounts/2`) shows a balance of **\$2200.00**, exactly \$1200 (initial credit) + \$1000 (transfer received) = \$2200. This cross-service verification closes the loop between Transaction and Account Services.

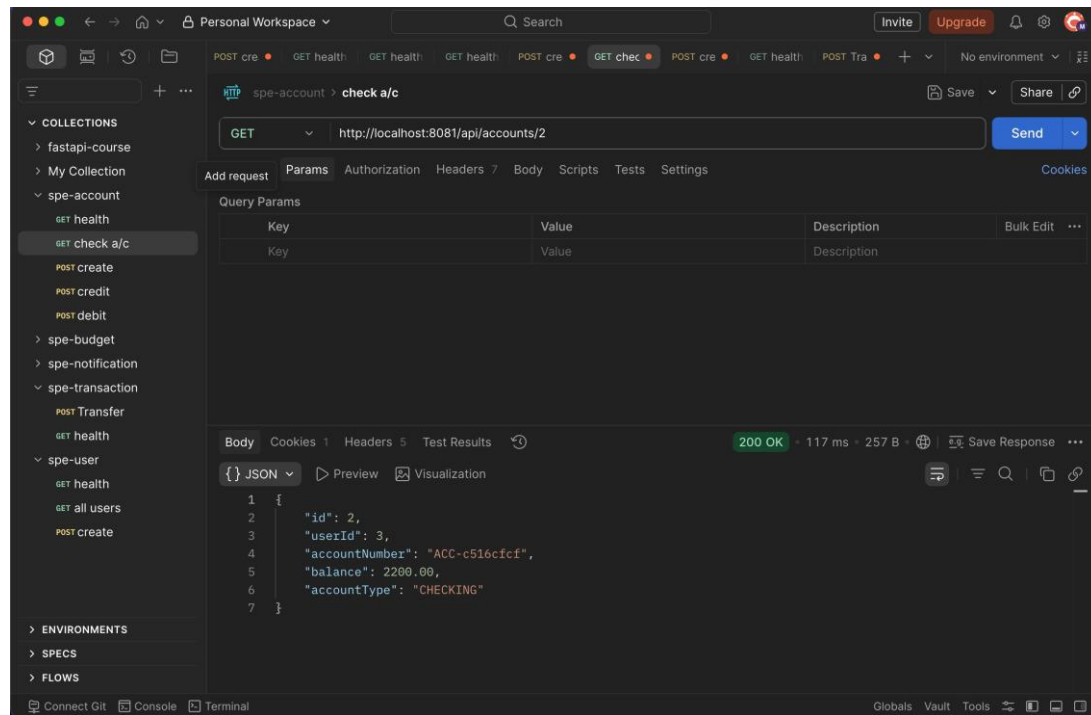


Figure 30: Account 2 balance verified at \$2200.00 after receiving a \$1000 transfer, confirming that the Transaction Service’s Saga correctly updated both account balances.

Budget Service — Create Budget and Track Spending

A POST to `/api/budgets` created a budget for `userId: 2` in the ENTERTAINMENT category with a `monthlyLimit` of **\$700.00**, `month: 3`, `year: 2026`, and an `alertThreshold` of **0.5** (i.e., alert when 50% of the budget is consumed). The service returned **201 Created** with the budget record including `currentSpending: 0` and a timestamped `updatedAt` field, confirming the record was persisted to PostgreSQL.

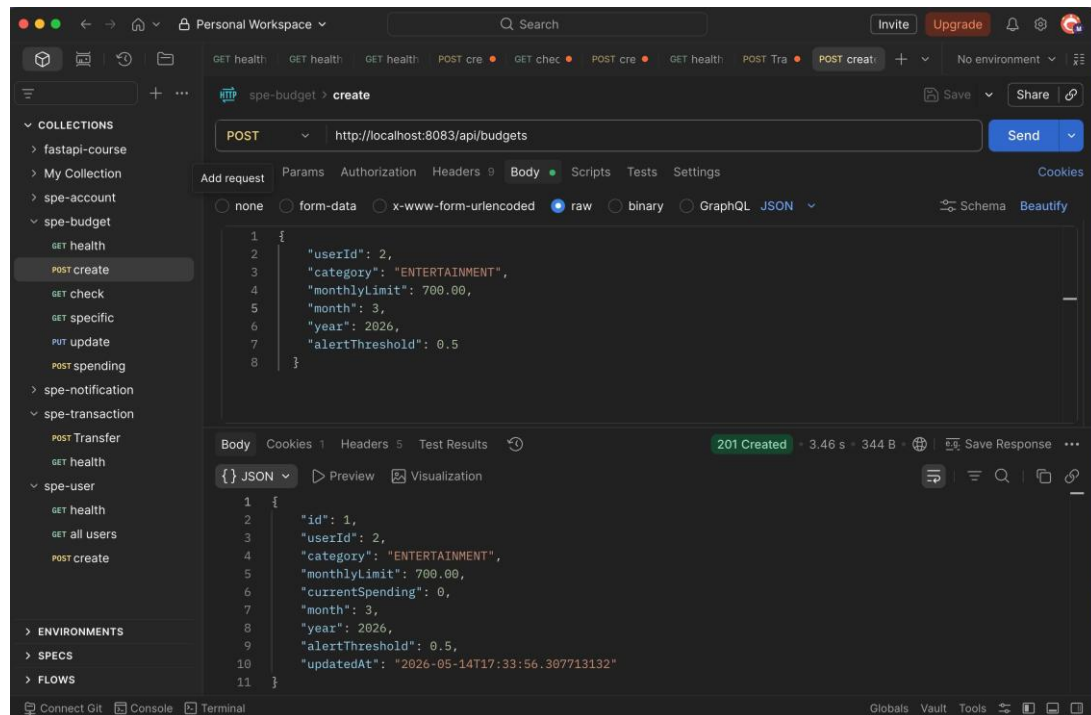


Figure 31: Budget Service: POST /api/budgets returns **201 Created** with the new budget including category, monthly limit, alert threshold, and initial zero spending.

To demonstrate the budget monitoring capability, a spending update of \$650 was posted to /api/budgets/1/spending. The response returned **200 OK** with currentSpending: 650, which is 92.9% of the \$700 monthly limit — well above the 50% threshold. The Budget Monitoring Service’s scheduler (running every 60 seconds) will detect this exceedance in the next cycle and call the Notification Service to dispatch a BUDGET_ALERT email, demonstrating event-driven cross-service communication.

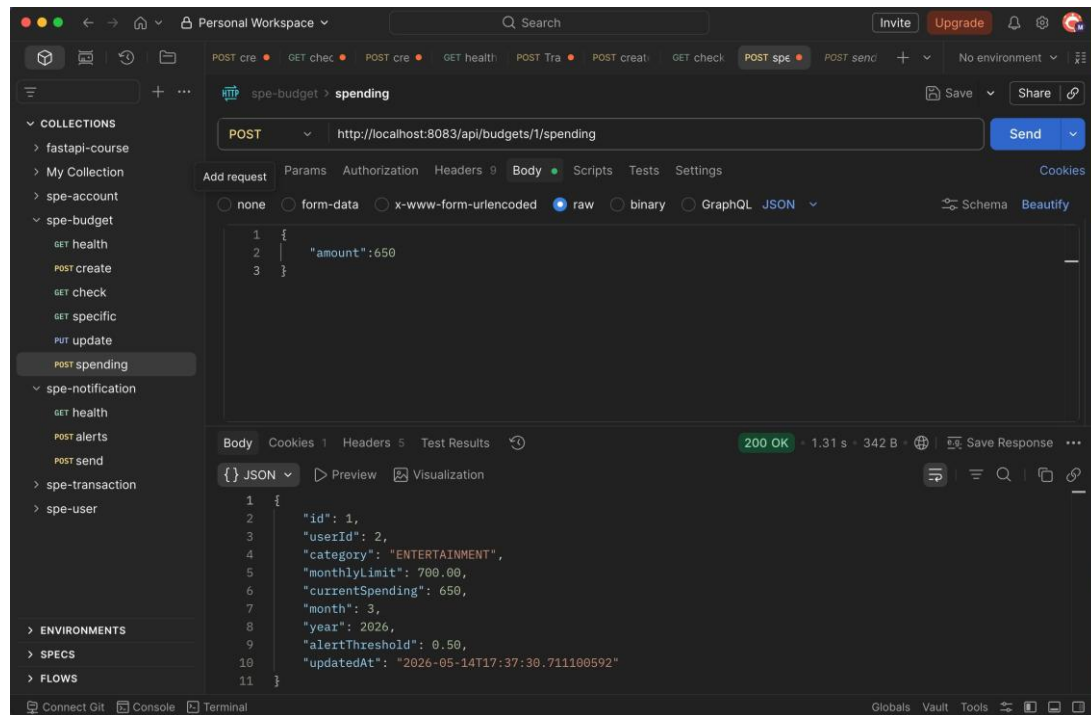


Figure 32: Budget Service: POST /api/budgets/1/spending updates current spending to \$650 against a \$700 limit, triggering the budget alert threshold check in the next monitoring cycle.

Notification Service — Send Budget Alert

To directly demonstrate the Notification Service's endpoint and the cross-service alert payload, a POST was made to /api/notifications with the exact request body that the BudgetMonitoringService generates when a threshold is exceeded: type: "BUDGET_ALERT", message: "You have used 85% of your ENTERTAINMENT budget for 3/2026", and channel: "EMAIL". The service returned **200 OK** in 37 ms, confirming the notification was accepted and logged. This validates the complete Budget → Notification inter-service communication chain and proves that the Notification Service is operational and reachable within the cluster on port 8084.

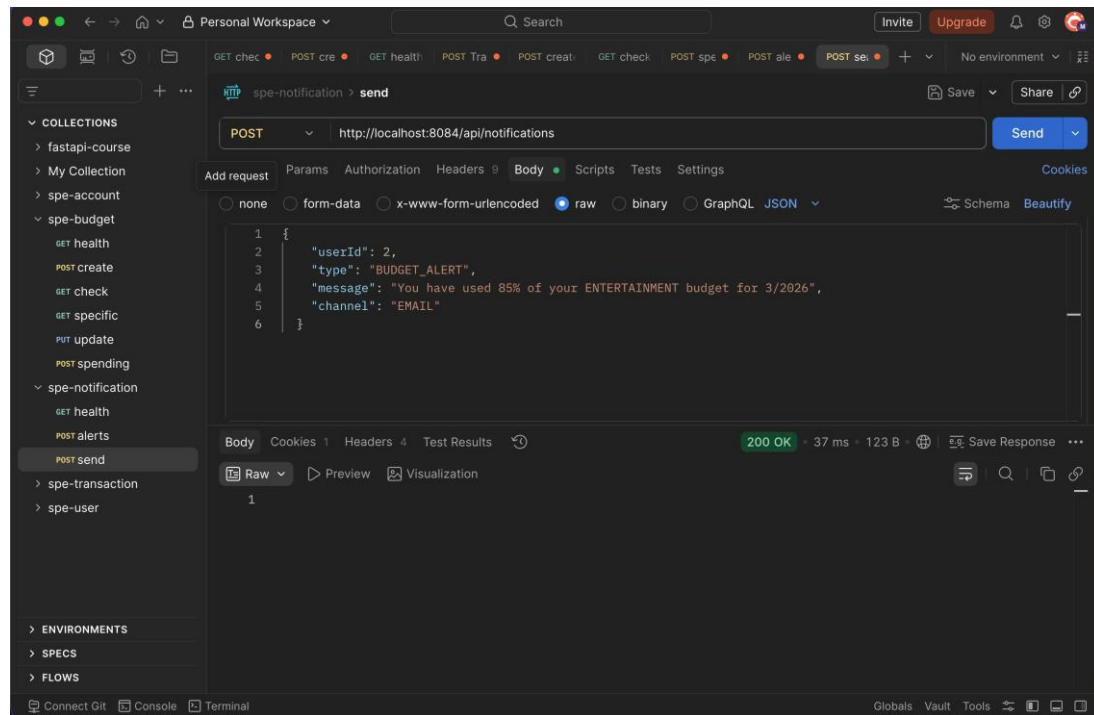


Figure 33: Notification Service: POST /api/notifications with a BUDGET ALERT payload returns **200 OK** in 37ms, confirming the service is live and the Budget-to-Notification alert pipeline is functional.

Section 10: ELK Stack — Observability and Log Aggregation

The ELK Stack (Elasticsearch, Logstash, Kibana) is deployed as a dedicated ArgoCD application (elk-app) into the logging Kubernetes namespace, managed by its own Helm chart in the GitOps repository. Elasticsearch stores log data in indices, Logstash receives structured JSON logs from all microservices over TCP port 5000 and routes them into per-service indices (springboot-{app}), and Kibana provides the web-based query and visualisation interface.

Each microservice uses the LogstashEncoder from the logstash-logback-encoder library in its logback-spring.xml configuration, which automatically serialises every log event as a JSON object containing the timestamp, log level, logger name, message, MDC fields (such as userId and transactionId), and the service tag. This structured logging approach makes it trivial to query and correlate events across services in Kibana.

The Kibana Discover view below shows the **budget-service** index live during the period when the Budget Monitoring Service was running its scheduled checks. The log stream includes:

- Repeated INFO entries from BudgetMonitoringService: *“Alert sent for budget ID: 1”* at one-minute intervals, demonstrating the scheduled alert dispatcher firing on schedule.
- An ERROR entry showing *“Failed to fetch transactions”* with the full error detail (HTTP 404 from the Transaction Service), demonstrating that errors are captured and searchable in Kibana with complete stack context.

- An INFO entry: *“Updating budget spending...”*, showing the second scheduled job (the five-minute budget reconciliation) running.

The index selector in the top-left reads **logs**, the time range is **Last 15 minutes**, and the histogram at the top shows log volume over time with a granularity of 30 seconds, confirming continuous log ingestion from the running pods. Available fields visible in the left panel (service, level, logger_name, message, env, thread_name) confirm that the structured log schema from the LogstashEncoder is being parsed and indexed correctly by Logstash.

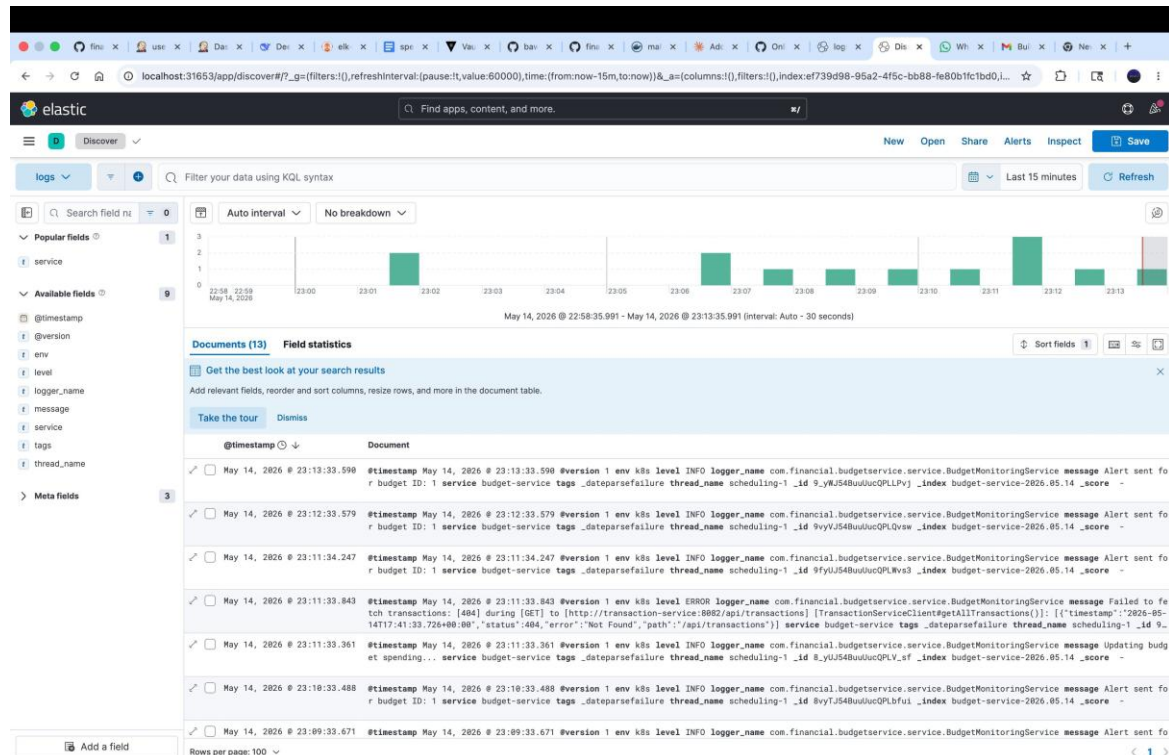


Figure 34: Kibana Discover view showing structured logs from the budget-service index: scheduled alert events, error traces, and budget update logs with full field extraction including service name, log level, logger, message, and timestamp.

Section 11: Jenkins Email Notification

Every Jenkins pipeline across all five services includes a post { always { mail(...) }} block in the Jenkinsfile. This means that regardless of whether the build succeeds or fails, Jenkins sends an email to mahendranath281201@gmail.com with the build result in the subject line and the Jenkins build URL in the body. This provides immediate feedback to the development team on every code push, without requiring anyone to log into the Jenkins UI.

The screenshot below shows the Gmail inbox with the email received after a successful User Service pipeline run (Build #38). The subject reads **“Build SUCCESS”** and the body contains the direct build URL: <https://pharmacopoeial-gracelynn-superstrong.ngrok-free.dev/>

which is the Jenkins server exposed via an ngrok tunnel for external access. The times-tamp (Wed 13 May, 11:20) aligns with the ArgoCD deployment history entry showing the *“Updated image tag to 38 in user-service”* commit deployed on the same date, completing the traceability chain from code push all the way through to a deployed, monitored, and notified system.

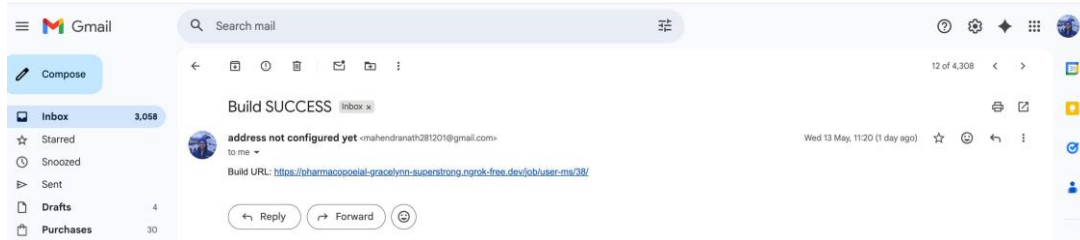


Figure 35: Gmail inbox showing the **“Build SUCCESS”** email notification sent automatically by Jenkins after a successful User Service pipeline run (Build #38), containing the direct build URL for traceability.